Intro to DP

SHSID CPC

Dictionary Definition

Dynamic Programming (DP) is an important algorithmic technique in Competitive Programming from the gold division to competitions like the International Olympiad of Informatics. By breaking down the full task into sub-problems, DP avoids the redundant computations of brute force solutions.

Fibonacci Numbers!

Let $F(0)=1$, $F(1)=1$, $F(2)=1$, and $F(x) = F(x - 1) + F(x - 2) + F(x - 3)$ for all $x \geq 3$. Find $F(n)$ % INT32_MAX for some $n \leq 1e8$. (If you don't know, INT32_MAX is a constant in C++ meaning "maximum number int32 can represent", stored in the <cstdint> or <bits/stdc++.h>)

Fibonacci Numbers!

Math kids will probably start doing a sort of recursion, then try to find a pattern... which will be hard to write on a computer.

Standard recursion: F(n) = F(n - 1) + F(n - 2) + F(n - 3) = F(n - 2) + F(n - 3) + F(n - 4) + F(n - 3) + F(n - 4) + F(n - 5) + F(n - 4) + F(n - 5) + F(n - 6)... and thus we get a O(3 ^ N) algorithm, which will “probably” timeout with N ≤ 1e8. It has a worse-case operation count of about 10 ^ 47712109 years... (I wasn’t able to make a comparison to anything, as even the number of subatomic particles in the observable universe and the time it takes for a human brain to break all the security codes on the planet were way too small)

Fibonacci Numbers!

And thus, DP or Dynamic Programming (not to be confused with *Displaced Person*)!

Its idea is fairly simple, probably seen by anyone who does math competitions: instead of recursion, we do something called *memoization*, which just means “putting things into a memo”, or storing calculated values so that recalculation is not needed.

In this case, we calculate F(x) from x = 3 to n, resulting in an O(n) time complexity.

Fibonacci Numbers!

```cpp
#include <bits/stdc++.h>
using namespace std;
int n;
int fib(int i){
    if(i < 3)return 1;
    return fib( i: i - 1) + fib( i: i - 2) + fib( i: i - 3);
}
int main() {
    cin >> n;
    cout << fib( i: n);
    return 0;
}
```

```cpp
#include <bits/stdc++.h>
using namespace std;
const int N = 100000005;
int n,dp[N];
int main() {
    cin >> n;
    dp[0] = dp[1] = dp[2] = 1;
    for(int i = 3; i <= n; i++){
        dp[i] = dp[i - 1] + dp[i - 2] + dp[i - 3];
    }
    cout << dp[n];
    return 0;
}
```

BEFORE ANY DP WORK: THINK WHAT STATES TO USE TO STORE WHAT KIND OF RESULT

https://atcoder.jp/contests/dp/tasks/dp_a Frog 1

A - Frog 1 Editorial

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 100 points

Problem Statement

There are N stones, numbered $1, 2, \ldots, N$. For each i ($1 \leq i \leq N$), the height of Stone i is h_i.

There is a frog who is initially on Stone 1. He will repeat the following action some number of times to reach Stone N:

- If the frog is currently on Stone i, jump to Stone $i+1$ or Stone $i+2$. Here, a cost of $|h_i - h_j|$ is incurred, where j is the stone to land on.

Find the minimum possible total cost incurred before the frog reaches Stone N.

Constraints

- All values in input are integers.
- $2 \leq N \leq 10^5$
- $1 \leq h_i \leq 10^4$

Input

Input is given from Standard Input in the following format:

$$
\begin{array}{l}
N \\
h_1 \quad h_2 \quad \ldots \quad h_N
\end{array}
$$

Output

Print the minimum possible total cost incurred.

Sample Input 1 Copy

```
4
10 30 40 20
```

Sample Output 1 Copy

```
30
```

If we follow the path $1 \rightarrow 2 \rightarrow 4$, the total cost incurred would be $|10 - 30| + |30 - 20| = 30$.

https://atcoder.jp/contests/dp/tasks/dp_a Frog 1

First, we can observe that the minimum cost at point x is only related to the costs of x - 1 and x - 2. So, we write f(x) where x is the current stone index:
f(x) = max(f(x-1) + |h_x - h_(x-1)|, f(x-2) + |h_x - h_(x-2)|).

This is really similar to Fibonacci numbers! Armed with a math formula, we can easily AC the problem (note that we have to process the edge case of x = 1, as x - 2 is negative).

```cpp
#include <bits/stdc++.h>
using namespace std;
int n,dp[100005],a[100005];
int main() {
    cin >> n;
    for(int i = 1; i < n; i++){
        dp[i] = 1e9;
    }
    for(int i = 0; i < n; i++){
        cin >> a[i];
    }
    for(int i = 1; i < n; i++){
        dp[i] = min(dp[i],dp[i-1]+abs(a[i]-a[i-1]));
        if(i > 1){
            dp[i] = min(dp[i],dp[i-2]+abs(a[i]-a[i-2]));
        }
    }
    cout << dp[n-1];
    return 0;
}
```

USACO 2017 January Contest, Gold

Problem 2. Hoof, Paper, Scissors

You have probably heard of the game "Rock, Paper, Scissors". The cows like to play a similar game they call "Hoof, Paper, Scissors".

The rules of "Hoof, Paper, Scissors" are simple. Two cows play against each-other. They both count to three and then each simultaneously makes a gesture that represents either a hoof, a piece of paper, or a pair of scissors. Hoof beats scissors (since a hoof can smash a pair of scissors), scissors beats paper (since scissors can cut paper), and paper beats hoof (since the hoof can get a papercut). For example, if the first cow makes a "hoof" gesture and the second a "paper" gesture, then the second cow wins. Of course, it is also possible to tie, if both cows make the same gesture.

Farmer John wants to play against his prize cow, Bessie, at N games of "Hoof, Paper, Scissors" ($1 \leq N \leq 100,000$). Bessie, being an expert at the game, can predict each of FJ's gestures before he makes it. Unfortunately, Bessie, being a cow, is also very lazy. As a result, she tends to play the same gesture multiple times in a row. In fact, she is only willing to switch gestures at most K times over the entire set of games ($0 \leq K \leq 20$). For example, if $K = 2$, she might play "hoof" for the first few games, then switch to "paper" for a while, then finish the remaining games playing "hoof".

Given the sequence of gestures FJ will be playing, please determine the maximum number of games that Bessie can win.

INPUT FORMAT (file hps.in):

The first line of the input file contains N and K.

The remaining N lines contains FJ's gestures, each either H, P, or S.

OUTPUT FORMAT (file hps.out):

Print the maximum number of games Bessie can win, given that she can only change gestures at most K times.

SAMPLE INPUT:

```
5 1
P
P
H
P
S
```

SAMPLE OUTPUT:

```
4
```

USACO 2017 January Contest, Gold
Problem 2. Hoof, Paper, Scissors

This is a very very standard DP problem.

For every DP problem, we need to first find a “DP state”, or basically the parameters of a math function.

Here, we can set a function f(n,p,k) denoting the maximum number of wins Bessie can have in the first n rounds, with p being her last gesture, and k being the number of times she has changed gestures.

Next, we need to find the recursive function rule (you all probably learned this in math class).

USACO 2017 January Contest, Gold
Problem 2. Hoof, Paper, Scissors

Our (compound) function rule will be:

$f(i,j,0) = f(i - 1,j,0) + (\text{does j beat FJ's i'th gesture})$
$f(i,j,x) = \max(f(i - 1,j,x),\max(f(i - 1,p,x - 1) + 1))$ for p in (gestures that are not j), with $1 \leq x \leq k$

All that's left is to write a for loop implementing this, and we're done.

Time complexity: O(N*K)
Space complexity: O(N*K)

USACO 2017 January Contest, Gold

Problem 2. Hoof, Paper, Scissors

```cpp
#include <bits/stdc++.h>
using namespace std;
const int N = 100005, P = 3, K = 21;
int n,k,fj[N],dp[N][P][K],ans;
char c;
int main() {
    ios::sync_with_stdio( sync: false);
    cin.tie( tiestr: nullptr);
    freopen("hps.in","r",stdin);
    freopen("hps.out","w",stdout);
    cin >> n >> k;
    for(int i = 1; i <= n ;i ++){
        cin >> c;
        fj[i] = (c == 'H')?0:(c == 'P')?1:2;
    }
    dp[1][(fj[1] + 1)%P][0]++;
    for(int i = 2; i <= n; i++){
        for(int j = 0; j < P; j++){
            bool add = ((fj[i] + 1) % P == j);
            dp[i][j][0] = dp[i - 1][j][0] + add;
            for(int x = min(i - 1,k);x; x--){
                if(add){
                    dp[i][j][x] = max( l: {dp[i - 1][j][x] + 1, dp[i - 1][(j + 1) % P][x - 1] + 1, dp[i - 1][(j + 2) % P][x - 1] + 1});
                }
                else{
                    dp[i][j][x] = dp[i - 1][j][x];
                }
            }
        }
    }
    for(int i = 0; i < P; i++){
        for(int j = 0; j <= k; j++){
            ans = max(ans,dp[n][i][j]);
        }
    }
    cout << ans;
    return 0;
}
```

Random Chinese Not-Very-Easy Problem: 关路灯

题目描述

复制 Markdown　展开　进入 IDE 模式

某一村庄在一条路线上安装了 n 盏路灯，每盏灯的功率有大有小（即同一段时间内消耗的电量有多有少）。老张就住在这条路中间某一路灯旁，他有一项工作就是每天早上天亮时一盏一盏地关掉这些路灯。

为了给村里节省电费，老张记录下了每盏路灯的位置和功率，他每次关灯时也都是尽快地去关，但是老张不知道怎样去关灯才能够最节省电。他每天都是在天亮时首先关掉自己所处位置的路灯，然后可以向左也可以向右去关灯。开始他以为先算一下左边路灯的总功率再算一下右边路灯的总功率，然后选择先关掉功率大的一边，再回过头来关掉另一边的路灯，而事实并非如此，因为在关的过程中适当地调头有可能会更省一些。

现在已知老张走的速度为 $1m/s$，每个路灯的位置（是一个整数，即距路线起点的距离，单位：m）、功率（W），老张关灯所用的时间很短而可以忽略不计。

请你为老张编一程序来安排关灯的顺序，使从老张开始关灯时刻算起所有灯消耗电最少（灯关掉后便不再消耗电了）。

输入格式

第一行是两个数字 n（表示路灯的总数）和 c（老张所处位置的路灯号）；

接下来 n 行，每行两个数据，表示第 1 盏到第 n 盏路灯的位置和功率。数据保证路灯位置单调递增。

输出格式

一个数据，即最少的功耗（单位：J，$1J = 1W \times s$）。

输入输出样例

输入 #1　复制

```
5 3
2 10
3 20
5 20
6 30
8 10
```

输出 #1　复制

```
270
```

说明/提示

样例解释

此时关灯顺序为 `3 4 2 1 5`。

数据范围

$1 \le n \le 50$，$1 \le c \le n$，$1 \le W_i \le 100$。

Random Chinese Not-Very-Easy Problem: 关路灯

First, let’s think of a naïve, slow, but definitely correct (ignoring TLE) solution: At each time period, we let Old Zhang go right or left. Of course, if we do this at every second, we can never hope for the program to end. Thus, we do a coordinate compression, meaning we do not iterate time, but we iterate the indices of the lamps.

Since we can choose to go right or left each time, our time complexity is O(2 ^ N), which is a bit too much.

Random Chinese Not-Very-Easy Problem: 关路灯

Then, let’s try to optimize everything with DP!

First of all, we need to find what our function acts upon. We notice that Old Zhang can only close an interval of lights (so he can’t close, for example, lights 1, 2, and 4). Thus, we can make the first two parameters of the function the leftmost lamp index and the rightmost lamp index.

Then, we see that we can not determine Old Zhang’s current location based on the first parameters alone! Then, another observation: he can only be at the endpoints of the interval (as he just closed a lamp). Thus, our function is complete!

Random Chinese Not-Very-Easy Problem: 关路灯

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
const int N = 55;
int n,c,dp[N][N][2],pre[N];
struct light{
    int d,p;
}lights[N];
signed main() {
    for(auto &x : long long &[55][2] : dp){
        for(auto &y : long long &[2] : x){
            y[0] = y[1] = 1000000000;
        }
    }
    cin >> n >> c;
    for(int i = 1; i <= n; i++){
        cin >> lights[i].d >> lights[i].p;
        pre[i] = pre[i-1] + lights[i].p;
    }
    dp[c][c][0] = dp[c][c][1] = 0;
    for(int len = 2; len <= n; len++){
        for(int i = 1; i + len - 1 <= n; i++){
            int j = i + len - 1;
            dp[i][j][0] = min(dp[i+1][j][1] + (lights[j].d-lights[i].d)*(pre[n]-pre[j]+pre[i]),dp[i+1][j][0] + (lights[i+1].d-lights[i].d)*(pre[n]-pre[j]+pre[i]));
            dp[i][j][1] = min(dp[i][j-1][0] + (lights[j].d-lights[i].d)*(pre[n]-pre[j-1]+pre[i-1]),dp[i][j-1][1] + (lights[j].d-lights[j-1].d)*(pre[n]-pre[j-1]+pre[i-1]));
        }
    }
    cout << min(dp[1][n][0],dp[1][n][1]);
    return 0;
}
```

The two very long, very ew lines are the “transition function” of our DP

Knapsack DP

An easy type of DP

Knapsack DP

Knapsack problems generally involve filling a limited container with a subset of items where we want to count or optimize some quantity associated with the items.

Dice Combinations:

CSES Problem Set

Dice Combinations

TASK | STATISTICS

Time limit: 1.00 s **Memory limit:** 512 MB

Your task is to count the number of ways to construct sum n by throwing a dice one or more times. Each throw produces an outcome between 1 and 6.

For example, if $n = 3$, there are 4 ways:

- $1 + 1 + 1$
- $1 + 2$
- $2 + 1$
- 3

Input

The only input line has an integer n.

Output

Print the number of ways modulo $10^9 + 7$.

Constraints

- $1 \leq n \leq 10^6$

Example

Input:
3

Output:
4

```cpp
#include <bits/stdc++.h>

using namespace std;

long long dp[1000001];

int main() {
    int n;
    cin >> n;
    dp[0] = 1;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= 6; j++) {
            if (i - j >= 0) { dp[i] += dp[i - j]; }
        }
        dp[i] %= 1000000007;
    }
    cout << dp[n] << "\n";
}
```

Minimizing Coins

CSES Problem Set

Minimizing Coins

TASK | STATISTICS

Time limit: 1.00 s **Memory limit:** 512 MB

Consider a money system consisting of n coins. Each coin has a positive integer value. Your task is to produce a sum of money x using the available coins in such a way that the number of coins is minimal.

For example, if the coins are $\{1, 5, 7\}$ and the desired sum is 11, an optimal solution is $5 + 5 + 1$ which requires 3 coins.

Input

The first input line has two integers n and x: the number of coins and the desired sum of money.

The second line has n distinct integers $c_1, c_2, \ldots, c_n$: the value of each coin.

Output

Print one integer: the minimum number of coins. If it is not possible to produce the desired sum, print -1.

Constraints

- $1 \leq n \leq 100$
- $1 \leq x \leq 10^6$
- $1 \leq c_i \leq 10^6$

Example

Input:
3 11
1 5 7

Output:
3

This is a classic knapsack:

- at each volume: think, how much i took to get there.
- dp[w] = the optimal (or smallest) number of coins to achieve a value of w

$$dp[w] = \min_{i=1}^{n} (dp[w - coins[i]]) + 1$$

Order matters

CSES Problem Set

Coin Combinations I

TASK | STATISTICS

Time limit: 1.00 s **Memory limit:** 512 MB

Consider a money system consisting of n coins. Each coin has a positive integer value. Your task is to calculate the number of distinct ways you can produce a money sum x using the available coins.

For example, if the coins are $\{2, 3, 5\}$ and the desired sum is 9, there are 8 ways:

- $2 + 2 + 5$
- $2 + 5 + 2$
- $5 + 2 + 2$
- $3 + 3 + 3$
- $2 + 2 + 2 + 3$
- $2 + 2 + 3 + 2$
- $2 + 3 + 2 + 2$
- $3 + 2 + 2 + 2$

Input

The first input line has two integers n and x: the number of coins and the desired sum of money.

The second line has n distinct integers $c_1, c_2, \ldots, c_n$: the value of each coin.

Output

Print one integer: the number of ways modulo $10^9 + 7$.

Constraints

- $1 \le n \le 100$
- $1 \le x \le 10^6$
- $1 \le c_i \le 10^6$

Example

Input:
3 9
2 3 5

Output:
8

Dynamic Programming

- Dice Combinations
- Minimizing Coins
- Coin Combinations I
- Coin Combinations II
- Removing Digits
- Grid Paths I
- Book Shop
- Array Description

...

dp[x] = number of ways (answer) for a goal of sum x.

dp[x] = sum dp[x-wi]

Order doesn't matter

CSES Problem Set

Coin Combinations I

TASK | STATISTICS

Time limit: 1.00 s **Memory limit:** 512 MB

Consider a money system consisting of n coins. Each coin has a positive integer value. Your task is to calculate the number of distinct ways you can produce a money sum x using the available coins.

For example, if the coins are $\{2, 3, 5\}$ and the desired sum is 9, there are 8 ways:

- $2+2+5$
- $2+5+2$
- $5+2+2$
- $3+3+3$
- $2+2+2+3$
- $2+2+3+2$
- $2+3+2+2$
- $3+2+2+2$

Input

The first input line has two integers n and x: the number of coins and the desired sum of money.

The second line has n distinct integers $c_1, c_2, \ldots, c_n$: the value of each coin.

Output

Print one integer: the number of ways modulo 10^9+7.

Constraints

- $1 \le n \le 100$
- $1 \le x \le 10^6$
- $1 \le c_i \le 10^6$

Example

Input:
3 9
2 3 5

Output:
8

Dynamic Programming

- Dice Combinations
- Minimizing Coins
- Coin Combinations I
- Coin Combinations II
- Removing Digits
- Grid Paths I
- Book Shop
- Array Description
- ...

We would be overcounting if using the last method.

dp[x] still denotes the answer for x, but now we consider for each coin, never returning to a traversed coin:

For a coin with value w:
for i from 1 to x
dp[i] += dp[i-w]

dp[x] is the answer.

Chinese Problems are easy: this is more classic in my opinion

P1048 [NOIP 2005 普及组] 采药

提交答案 加入题单 复制题目

题目描述

复制 Markdown 展开 进入 IDE 模式

辰辰是个天资聪颖的孩子，他的梦想是成为世界上最伟大的医师。为此，他想拜附近最有威望的医师为师。医师为了判断他的资质，给他出了一个难题。医师把他带到一个到处都是草药的山洞里对他说：“孩子，这个山洞里有一些不同的草药，采每一株都需要一些时间，每一株也有它自身的价值。我会给你一段时间，在这段时间里，你可以采到一些草药。如果你是一个聪明的孩子，你应该可以让采到的草药的总价值最大。”

如果你是辰辰，你能完成这个任务吗？

输入格式

第一行有 2 个整数 T（$1 \leq T \leq 1000$）和 M（$1 \leq M \leq 100$），用一个空格隔开，T 代表总共能够用来采药的时间，M 代表山洞里的草药的数目。

接下来的 M 行每行包括两个在 1 到 100 之间（包括 1 和 100）的整数，分别表示采摘某株草药的时间和这株草药的价值。

输出格式

输出在规定的时间内可以采到的草药的最大总价值。

输入输出样例

输入 #1 复制

```
70 3
71 100
69 1
1 2
```

输出 #1 复制

```
3
```

说明/提示

【数据范围】

- 对于 30% 的数据，$M \leq 10$；
- 对于全部的数据，$M \leq 100$。

【题目来源】

NOIP 2005 普及组第三题

DP[time] = best total value at using exactly time time

range of time: 1000

DP initialized to negative inf

DP[0] = 0

At each medicine, consider for each DP[time] (in reverse order!):

- Take it: DP[time-w] + w
- Don’t take it: DP[time]

Another Chinese easy problem

P1060 [NOIP 2006 普及组] 开心的金明

提交答案 加入题单 复制题目

题目描述

复制 Markdown 展开 进入 IDE 模式

金明今天很开心，家里购置的新房就要领钥匙了，新房里有一间他自己专用的很宽敞的房间。更让他高兴的是，妈妈昨天对他说："你的房间需要购买哪些物品，怎么布置，你说了算，只要不超过 N 元钱就行"。今天一早金明就开始做预算，但是他想买的东西太多了，肯定会超过妈妈限定的 N 元。于是，他把每件物品规定了一个重要度，分为 5 等：用整数 $1-5$ 表示，第 5 等最重要。他还从因特网上查到了每件物品的价格（都是整数元）。他希望在不超过 N 元（可以等于 N 元）的前提下，使每件物品的价格与重要度的乘积的总和最大。

设第j件物品的价格为 v_j，重要度为 w_j，共选中了 k 件物品，编号依次为 $j_1, j_2, \ldots, j_k$，则所求的总和为：

$$v_{j_1} \times w_{j_1} + v_{j_2} \times w_{j_2} \ldots + v_{j_k} \times w_{j_k}$$

请你帮助金明设计一个满足要求的购物单。

输入格式

第一行，为 2 个正整数，用一个空格隔开：n, m（$n < 30000, m < 25$）其中 n 表示总钱数，m 为希望购买物品的个数。

从第 2 行到第 $m+1$ 行，第 j 行给出了编号为 $j-1$ 的物品的基本数据，每行有 2 个非负整数 v, p（其中 v 表示该物品的价格（$v \le 10000$），p 表示该物品的重要度（$1 \le p \le 5$）。

输出格式

1 个正整数，为不超过总钱数的物品的价格与重要度乘积的总和的最大值（< 100000000）。

输入输出样例

输入 #1 复制

```
1000 5
800 2
400 5
300 5
400 3
200 2
```

输出 #1 复制

```
3900
```

说明/提示

NOIP 2006 普及组 第二题

DP[cost] best objective function achieved with cost cost

for each item:
for cost from n to 1:
DP[cost] = min(DP[cost], DP[cost-vi]+vi*wi).

Another Chinese not too easy problem

P5020 [NOIP 2018 提高组] 货币系统

提交答案 加入题单 复制题目

题目背景

复制 Markdown 展开 进入 IDE 模式

NOIP2018 提高组 D1T2

题目描述

在网友的国度中共有 n 种不同面额的货币，第 i 种货币的面额为 $a[i]$，你可以假设每一种货币都有无穷多张。为了方便，我们把货币种数为 n、面额数组为 $a[1..n]$ 的货币系统记作 (n, a)。

在一个完善的货币系统中，每一个非负整数的金额 x 都应该可以被表示出，即对每一个非负整数 x，都存在 n 个非负整数 $t[i]$ 满足 $a[i] \times t[i]$ 的和为 x。然而，在网友的国度中，货币系统可能是不完善的，即可能存在金额 x 不能被该货币系统表示出。例如在货币系统 $n = 3, a = [2, 5, 9]$ 中，金额 $1, 3$ 就无法被表示出来。

两个货币系统 (n, a) 和 (m, b) 是等价的，当且仅当对于任意非负整数 x，它要么均可以被两个货币系统表出，要么不能被其中任何一个表出。

现在网友们打算简化一下货币系统。他们希望找到一个货币系统 (m, b)，满足 (m, b) 与原来的货币系统 (n, a) 等价，且 m 尽可能的小。他们希望你来协助完成这个艰巨的任务：找到最小的 m。

输入格式

输入文件的第一行包含一个整数 T，表示数据的组数。

接下来按照如下格式分别给出 T 组数据。每组数据的第一行包含一个正整数 n。接下来一行包含 n 个由空格隔开的正整数 $a[i]$。

输出格式

输出文件共有 T 行，对于每组数据，输出一行一个正整数，表示所有与 (n, a) 等价的货币系统 (m, b) 中，最小的 m。

输入输出样例

输入 #1 复制

```
2
4
3 19 10 6
5
11 29 13 19 17
```

输出 #1 复制

```
2
5
```

说明/提示

在第一组数据中，货币系统 $(2, [3, 10])$ 和给出的货币系统 (n, a) 等价，并可以验证不存在 $m < 2$ 的等价的货币系统，因此答案为 2。在第二组数据中，可以验证不存在 $m < n$ 的等价的货币系统，因此答案为 5。

【数据范围与约定】

对于 100% 的数据，满足 $1 \le T \le 20, n, a[i] \ge 1$。

If some coin's value is the sum of some combination of other coins' values, then that coin is useless!

Others are all necessary.***

Sort the a's ascendingly:

- DP[i] denotes whether i is achievable with the system.
- cnt = n
- DP[0] = 1, others all 0
- then go up in a:
  - if DP[current a] = 1:
    - this a is useless, cnt--, skip.
  - Update DP